Rolling Chat Contractor Specification Draft

Product and engineering brief | 11 October 2026

Build rolling.chat as a public discussion community with AI assistance. People should browse without a wallet, post anonymously or under privately owned pseudonyms, discover subboards, and collaborate with Bok inside a thread. The result should feel like a social application rather than a blockchain control panel.

The intended engagement is eight weeks. Deliver the core platform, the expanded Bok plugin, reproducible deployment tooling and acceptance evidence. This draft is suitable for technical scoping and bids; proposed defaults on the final page must be priced explicitly.

Starting implementation

The application repository is aztec_experiments. As inspected on 11 October, master is at 458724c. Remote proving and board plugins were merged on 8 October; the application is now pinned to Aztec 6.0.0-rc.1.

When developing the application, use a local devnet with proving disabled for faster iteration. Local devnet transaction flows should identically mirror testnet flows. Project acceptance is dependent on validating the application works as intended on testnet with real wallet flows.

The project describes work at the smart contract layer, frontend work and backend work across multiple backends (prover server, plugin server, rolling.chat backend server). Attempt to program to interfaces and not implementations - software components should have clear boundaries interfaced with via APIs so that entities can work in parallel each on their own component while minimising breaking changes.

The current application is running on testnet with 0 users. You are free to update the application without a migration plan, it is not yet live.

Existing component	Baseline to preserve
Board and moderation	Public messages; private collateral-backed posting; rate limits; moderator rules and flags.
Wallet and funding	Ethereum wallet connection, passkey-backed Aztec account, private fee funding, transaction journals and recovery.
Remote prover	Hosted proof generation with browser verification; local proving choice and explicit failures.
Plugins and Bok	Registered bot replies; per-plugin USDC escrow; measured inference billing; repository tools.
Public reading	Wallet-free feed and board discovery; incremental chain indexing.

Qualification boundary

The V6 migration record includes real-proof testnet posting, a paid linked Bok reply, withdrawals and Ethereum refunds. It retains failed runs and reconciliation boundaries.

Discussion interface and social features

Prioritise desktop. Use Reddit as the reference for navigation and readable organisation, Discord for visual atmosphere, and X for the feed interaction. Produce an original design system and clickable prototype before implementing the complete interface. Mobile should retain usable core journeys without dictating the desktop layout.

Required screens

- Home feed; top-level board; subboard; all-subboards directory and subboard search.
- Thread view with replies to individual posts, authenticated Bok replies and links to generated pages or the thread repository.
- Composer with identity dropdown, formatting preview, content allowance and clear transaction progress.
- Global identity profile, registered-name management, bookmarks, followed boards and identities, and notifications.
- Single onboarding flow; balances and withdrawals; account backup and recovery; moderator controls.

Feed and posting

The feed must support continuous scrolling, stable post links, opening a thread and returning to the previous scroll position. Avoid duplicates and disruptive jumps when new posts arrive. Provide accessible loading, empty, pending, removed-content and failure states. A chronological Following feed and a deterministic Home ranking are proposed defaults; final ranking is a week-one design deliverable.

Users can reply to posts in a Twitter-style thread. Every reply has a canonical parent and root-thread identity. Posts cannot be edited or hidden by their authors. Moderation can hide content from the application, but cannot erase public chain records. Removed-parent treatment must preserve navigation without revealing the removed text.

Keep the existing 992-byte UTF-8 message ceiling. Present a character-oriented allowance in the composer that correctly accounts for multibyte input. Never advertise a universal 992-character allowance or truncate a submitted post silently. URLs and labelled links must render safely; permit only supported link schemes and treat posted markup as untrusted data.

Social interactions

Launch scope includes likes, quote posts, reposts, bookmarks, following boards and identities, and reply notifications. Likes must be recorded onchain. All persistent core social state must be recoverable from chain records, excluding private bookmarks, follows and notification preferences. Notification delivery and feed ranking are derived services, not new sources of authority.

The contractor must propose how likes are counted and duplicates prevented without exposing the owning account or connecting its different identities. The proposed default is one active like per underlying account per post, regardless of the display name used. No direct messaging or advertising system is included in this scope.

Boards and moderation

The rolling.chat frontend launches with one curated top-level board. Other operators may deploy independent boards but must host their own frontend and server. The rolling.chat homepage does not automatically include their boards.

Subboard creation and discovery

Anyone with an account can create a subboard under the curated board, paying transaction fees. Posting collateral is not a condition of creation. A new subboard is usable immediately. Its creation, rules and identity must be authenticated by chain records.

Subboards have no further nested subboards and cannot be renamed, archived or transferred by their creators. Moderator rejection is a separate moderation action. Ordinary creation must not add a subboard to the default homepage automatically. Users find subboards through search and a directory listing all existing subboards, with clear rejected-state handling.

A subboard name, description, immutable identifier and discoverable link are needed. Case handling and collision rules are proposed in the final-page defaults. Display information and creation status must be reconstructible from chain history.

Effective rules

A subboard applies the set union of main-board rules and its own additional rules. Main-board changes propagate to subboards. Separate subboard-creation rules prohibit additions that contradict or undermine the main-board policy. A moderator may reject a creation after publication if it violates those creation rules.

Policy interpretation must preserve the main rules even if local text contains instructions to disregard them. Treat submitted rules as policy data, not privileged instructions to a model. Store policy versions and moderation outcomes so the current effective rules and the version used for a decision can be inspected.

Moderation interface and recovery

- Show current effective rules before posting and provide a clear way to inspect a subboard rejection and its reason.
- Record authorised rejection and post-removal outcomes onchain. Rebuilding the index must reproduce them.
- Apply existing moderation to human messages and Bok board replies. Rejecting or removing a message must not expose private author ownership.
- Retain recoverable account funds and existing withdrawal paths when a subboard is rejected.

Proposed rejection behavior: retain a visible tombstone in the directory, hide its content from ordinary browsing and stop new posts there. Parent-moderator authority covers all subboards. Updating local rules, local moderator roles and review appeals are proposed decisions to settle during week one; they are not permissions granted implicitly to creators.

Usernames profiles and onchain avatars

A user can register multiple global usernames under one private account. Each consists of a first name and a last name, and the pair must be unique. John Doe and Jane Doe do not collide. Offer random name generation and verify availability before registration; the transaction must enforce uniqueness under concurrent registrations.

Ownership and identity selection

The composer dropdown lists all names owned by the user, plus Anonymous. Named posts authenticate the selected name. Anonymous posts display exactly Anonymous and publish no reusable author identifier. The public interface must not expose which different names belong to the same account.

Names cannot be retired. Existing ownership must survive account restoration using the user recovery keys. The name registry must not publicly map several names to one wallet. Authentication and uniqueness must be proved without revealing that grouping. A stable pseudonym intentionally links its own named posts, but must not link those posts to other pseudonyms or anonymous activity.

Proposed name defaults: trim whitespace, collapse repeated spaces and compare names case-insensitively; keep the displayed form readable. Names remain immutable in the first release. The implementation must specify supported characters, normalisation, reserved bot names and maximum lengths before deployment.

Profiles

Each username has a profile page with its named-post history, a bio and avatar. Updating the avatar updates its presentation on earlier posts. Public profiles must not list other names owned by the account. A profile must distinguish authenticated bot identities from human names and unverified mentions.

Avatar storage

Accept compressed uploaded raster images, cropped or resized for a 100 by 100 pixel presentation. Cap the encoded image at 8,192 bytes. Publish the actual bytes in authenticated onchain logs and keep their content hash and current-version reference in profile state. External hosting alone does not meet the requirement.

The installed V6 constants retain 63 application public-storage updates per transaction, a 4,096-field aggregate public-log payload and an 8,475-field transaction blob-data ceiling. At 31 bytes per packed field, an 8 KiB image needs 265 payload fields before metadata. This supports the proposed representation structurally; gas, latency, image quality and archival retrieval still require a measured prototype.

Compress automatically and preview the final image before publishing. Strip unnecessary metadata, enforce safe decoding and apply avatar moderation. Maintain archival retrieval so a clean backend can reconstruct the current image. Historical chain publication does not guarantee every public RPC retains image data indefinitely.

Funding and account onboarding

There is no fee sponsorship. A new user starts with an existing Web3 wallet holding ETH or USDC. The experience must minimise approvals and manual steps while funding board admission, transaction fees and Bok use. Preserve noncustodial ownership and existing withdrawal rights.

Required user experience

- Connect the wallet, create or unlock the private account, choose a funding amount and inspect a simple cost breakdown.
- Submit one initial funding transaction where the supported wallet and asset route permit it.
- Automatically complete bridge claims and Aztec setup once ready, with a single understandable progress view.
- Resume after reload, wallet interruption or temporary service failure without duplicate payments or a manual claim workflow.
- Explain the result as ready to post, still setting up or requiring a specific action. Never report success while funds remain unclaimed.

Engineering scope

Treat this as a hard engineering milestone. Prototype an aggregate funding route, asset conversion if needed, token approval handling and private fee bootstrap. Existing collateral, fee credit and plugin escrow are separate economic purposes; the new interface must explain them without requiring users to understand several unrelated funding applications.

Ethereum-to-Aztec delivery is asynchronous. A single Ethereum transaction can initiate funding, while later Aztec transactions complete it automatically. Do not promise synchronous atomic completion across both chains. Document which Ethereum operations are atomic and how each later stage is recovered.

USDC-only onboarding must resolve how gas is paid without app-funded sponsorship. Token permits, wallet batching and account capabilities vary; their availability must be verified with actual target wallets. Unsupported routes must be identified before taking funds. The named-wallet target is MetaMask and Phantom in Ethereum mode, with a tested compatibility matrix and clear chain selection.

Acceptance and cost

Demonstrate both ETH-origin and USDC-origin journeys, record wallet prompts and transaction counts, and identify any route that cannot satisfy the one-initial-transaction target. A technical impossibility or wallet limitation must be presented with a concrete alternative for owner decision; it is not permission to silently dilute the requirement.

Separate refundable collateral, chain fees, conversion costs and Bok credit. Show available and reserved balances, withdrawal conditions and measured costs. Additional funding must follow the same recovery discipline. Keep encrypted recovery material on the user side and verify it survives supported restoration flows.

Optimisation must preserve unlinkability. Combining funding into a convenient transaction must not later expose that funding account as the author of several usernames, posts or likes. Existing V6 wallet and fee qualification is a foundation, not acceptance of this new aggregate route.

Privacy and recoverable backend architecture

Anonymous posts must be cryptographically unlinkable to the account, its usernames and its other anonymous posts. Outsiders must not connect different usernames to one account. Aim to preserve those guarantees against rolling.chat backend and node operators as well as public chain observers. The prover operator may be trusted with private witnesses; anonymity against that service is not required.

Privacy design and evidence

Publish an explicit observer and trust model. Analyse public transactions, note and nullifier construction, fee refunds, username registration, likes, follows, notifications, Bok payments, index queries and account restoration. The current plugin account-to-request disclosure must be replaced before an anonymous Bok flow can satisfy this specification.

Avoid sending account identity or the complete private name list to the cache API. Prevent private subscriptions or account-specific queries from becoming an easy node-side link between identities. Any batching, proxying, local processing or private query approach must have a stated guarantee and be tested. Merely removing an address from the page is insufficient.

Cryptographic unlinkability cannot prevent self-identification in text or guarantee anonymity against arbitrary writing-style, network or timing analysis. Describe these residual observations precisely. Do not claim that the backend/node privacy objective is achieved if request traces still reveal the relationships it is intended to hide. Independent expert review is required for assurance claims.

Chain authority and indexing

All persistent core application state must be recoverable by scanning authenticated onchain records. This includes boards, rules, moderation, posts and threads, names and profiles, avatars, likes, reposts, quote links, and encrypted user preferences such as bookmarks and follows. User recovery keys are required to decrypt private state; scanning cannot recreate lost secrets.

The backend indexes and caches this data for feeds, search and notifications. It cannot become the sole owner of a relationship, moderation result or balance. Ranking and search indexes may be derived; their inputs and algorithms must be recoverable. Operational secrets and running service sessions are provisioned separately.

Plan for an operated Aztec node and durable archiver, subject to measured capacity and cost. Use incremental ingestion, bounded requests, pagination, shared caches and reorg handling even with a dedicated node. Specify how public content and encrypted records are retained for a complete rebuild.

Plugin boundary

Bok repositories, artifact source, conversation summaries and other plugin workspace data belong to the Bok server. They are excluded from core onchain recovery. Bok board messages and authenticated reply links remain core chain data. A cache rebuild must succeed even when the Bok server is unavailable. Plugin outage must not break ordinary reading or posting.

Bok thread workspace and autonomous coding

The contractors must deliver the Bok plugin as well as the platform. Keep model choices, budget policy, summarisation thresholds and runner details inside the plugin. The core understands authenticated invocations and replies; it must not hard-code one provider or conversation strategy.

Conversation and workspace

Maintain one persistent workspace and one repository per thread. Independent threads run concurrently. Within a thread, serialise conflicting repository changes and record the revision each invocation reads and produces. Service restarts must preserve accepted workspace changes without repeating uncertain paid calls.

The entire thread conversation must be available to the agent, including earlier Bok calls and replies. When the context is too large or expensive, use a cheaper model to summarise the preceding conversation up to, but excluding, the latest invoking message. Give the main model that summary plus the latest message. Support retrieval of original messages when needed.

Tune this approach against representative long threads. Evaluate whether summaries retain instructions, decisions, unresolved issues and repository references. Measure cost and latency, and account for summarisation as paid plugin work. Thresholds and model choices are implementation decisions governed by user experience, not platform parameters.

Participation and autonomy

Anyone may instruct Bok. The first substantive request may establish expectations that later context carries forward, but this creates no formal owner permission. Do not explain implicit thread rules or an anarchic permission model in product copy. Preserve normal context awareness while treating conversation and repository content as untrusted input.

Bok may create its thread repository, edit and test code, commit accepted changes and automatically publish static artifacts. The setup must also support connected existing repositories, subject to their explicit permissions. Open thread participation cannot grant authority beyond the repository permission configured by its operator.

GitHub and execution safety

Use a rolling.chat-controlled organisation and a dedicated GitHub App identity with scoped installation credentials. Do not use the founder's personal token as the bot identity. Automate thread repository provisioning through an operator-controlled broker and keep repository administration separate from model tools.

Use isolated execution for code and builds; keep chain signing, provider, hosting and GitHub secrets outside model-controlled processes. Thread projects may allow automatic commits and publishing. Existing repositories retain configured branch protections and approval policies. Do not expose a general mainnet wallet or unrestricted deployment credential to Bok.

Preserve per-plugin balances, measurable charges and unused-credit release. Put limits on execution, requests and storage; tune them within the plugin rather than prompting the owner to prescribe token budgets. No failed or uncertain paid invocation may silently repeat.

Static artifacts and independent operation

Bok can generate static HTML, CSS and JavaScript pages and return a shareable link in its board reply. Host the latest published version indefinitely and offer a source download. Version history belongs to the plugin workspace; previous versions do not have to remain deployed at public URLs.

Publishing and isolation

- Give a thread artifact a stable latest-version link and record which repository revision produced the published content.
- Publish a complete version atomically so readers do not receive a mix of old and new files.
- Serve generated content on a separate origin with no board wallet data, authentication cookies or privileged credentials.
- Allow public API calls and external assets. Requests that need secrets must use a separately scoped plugin facility rather than embedding secrets in downloadable source.
- Provide source downloads of the current published artifact and preserve internal project history for recovery and further edits.

Define abuse controls, storage accounting, backups and artifact moderation in the plugin operator runbook. Indefinite retention is the product requirement, not a promise of cost-free hosting. Removing a board message cannot retract an external request or undo a repository change; hosting removal is a distinct plugin action.

Independent board deployment skill

Deliver an AI skill and supporting commands that let another operator deploy an independent board, frontend and server with minimal manual work. Do not assume access to rolling.chat credentials or infrastructure. The skill must explain expected costs, supported chain/version, domain and account prerequisites, funding, authority roles and backup responsibilities.

The skill should guide configuration, reproducible builds, contract and portal deployment, fee routing, moderator setup, frontend publication, node/indexer setup, optional remote proving, and optional Bok installation. Include exact-transaction recovery, verification before accepting funds, monitoring, credential rotation and rollback instructions.

Provision secrets outside source control. Require the operator to authorise paid infrastructure and transactions explicitly. The skill must not copy a test configuration into production or represent a successful setup as an independent audit.

Handover

Provide source code, dependency locks, build and deployment commands, configuration schemas, service definitions, tests, a chain-data schema and rebuild procedure. Provide separate core and Bok recovery runbooks, including repository and artifact backups. Name the operational roles that must be assigned for the board, moderator, node, prover and Bok; hosting autonomy does not remove those responsibilities.

Demonstrate a clean independent deployment using fresh disposable identities and a fresh frontend origin. The skill is accepted only when another operator can follow it without hidden state from the contractors' machines.

Acceptance scenarios for the core platform

Attach each implemented requirement to an executable scenario and source revision. Use real proofs for proof-sensitive acceptance, real wallet interactions for onboarding and exact accounting for funds. Mocked models are useful for interface checks but do not establish Bok quality or real billing.

Social and moderation journeys

- An unfunded reader browses the main feed, searches subboards, opens a thread and returns to the previous scroll position without a wallet.
- An account without posting collateral creates a subboard using transaction fees. It is immediately usable and appears in search and the directory, without automatic homepage promotion.
- A moderator rejects contradictory subboard rules afterwards. The rejection reason survives a cache rebuild, new posting is handled consistently and account funds remain recoverable.
- Parent-policy changes propagate to a subboard, and both human posts and Bok replies are evaluated under the recorded effective rules.
- Two concurrent registrations cannot acquire the same normalised full name. Different names owned by one account reveal no public ownership grouping.
- Named and Anonymous posts, nested replies, quote posts, reposts and onchain likes render consistently across feed, thread and profile.
- Oversized image and text inputs are rejected before spending. Valid 8 KiB avatars and current-profile updates survive index deletion and rebuild.

Funding privacy and recovery

- Fresh-wallet ETH and USDC onboarding records every prompt, transaction and funding allocation. Ready means the user can actually post and invoke Bok.
- Interrupt each funding stage, reload and restore the account. The application reconciles completed actions and never charges again merely because a receipt response was lost.
- An account can withdraw eligible collateral and available plugin credit with exact accounting, including refusal of replay and wrong recipients.
- Inspect public effects and backend/node request traces for links across usernames, anonymous posts, likes and Bok requests. Provide protocol reasoning and reviewer disposition in addition to trace tests.
- Delete the core database and rebuild from chain records and retained archive data. Restore private bookmarks, follows and preferences with user keys. Repeat across a simulated reorg.
- Core reading and posting remain available with Bok offline. Node/provider outages produce honest states and preserve recovery records.

Performance and accessibility

Week one must set measured targets for initial feed load, thread load, scroll responsiveness, search, index freshness, rebuild duration, node requests, proof latency and running costs. Report cold and warm results with hardware, network and dataset size. Test keyboard navigation, focus, readable contrast and screen-reader status announcements. Targets cannot be chosen retrospectively merely to fit observed results.

Bok acceptance and delivery plan

Plugin acceptance

- Two users invoke Bok in the same thread; both changes arrive in one persistent repository in the defined order. Two independent threads progress concurrently without state leakage.
- A long conversation triggers the cheaper-model summary stage. The main model uses the summary and latest message and can retrieve the original context. Quality and measured cost are recorded.
- A user can request code, tests and a static page; Bok publishes it automatically, returns its link and provides matching downloadable source. A later request updates the same workspace and latest link.
- Restart the plugin after a committed change and after an uncertain paid call. Accepted code persists and the uncertain call is not silently replayed.
- Actual inference charges match the metering records and escrow settlement. Unused reserves are released under documented completion and expiry behavior.
- Generated code and pages cannot access board credentials, another thread workspace or unconfigured repositories. Paid external APIs cannot create unbounded liability.

Proposed eight week milestones

Period	Reviewable output
Week 1	Baseline audit, design prototype, data and privacy design, funding feasibility experiment, cost and performance targets.
Weeks 2 and 3	Vertical slice: onboard, choose identity, create subboard, post, reply, like and rebuild the index. Early privacy review.
Weeks 4 and 5	Full social interface, moderation, profiles/avatars, feeds/search and persistent concurrent Bok workspaces.
Weeks 6 and 7	Static artifacts, GitHub App provisioning, independent deployment skill, recovery and load/failure qualification.
Week 8	Integrated candidate, defect closure, final acceptance evidence, runbooks and operator handover.

Review feasibility in week one and disclose cost or schedule conflicts. Preserve requirements unless the owner agrees a change. Independent audit, network clearance and 14-day soak remain production gates; mainnet deployment needs separate authorisation.

Team and commercial response

One product/frontend engineer owns design and user journeys. One Aztec/backend engineer owns contracts, privacy, indexing and Bok infrastructure. Both own integration. Demonstrated Aztec and privacy expertise is required for the second role.

Quote against the USD 15,000 total budget target. State whether infrastructure, model usage, independent review and operation are included. Extra budget is not authorised. Propose milestone payments tied to reviewable outputs.

Proposed defaults

Confirmed requirements in the preceding sections come from the owner's decisions. The items below are proposed working defaults or remaining details, to be resolved with a concrete design or feasibility result during week one. They do not authorise scope reduction.

Defaults

- Home has a deterministic ranked feed; Following has chronological order. Return-to-feed preserves position. Specify ranking inputs and how subboards enter Home without automatic creation promotion.
- Subboard identifiers use an immutable unique slug. Rejecting a subboard leaves a directory tombstone and blocks new posting. Determine whether local rules may later be tightened and whether moderator rejection is reversible.
- One active like per private account per post, across its usernames; unlike is supported. Use a privacy-preserving design and expose counts without an account membership list.
- Full names compare case-insensitively after whitespace normalisation. Names remain immutable, and bot handles are reserved. Specify Unicode and collision behavior.
- Reply notifications are in-app. Define private follows, bookmark recovery, read markers and subscription privacy before introducing email or push delivery.
- Thread project commits and static publishing are automatic. Connected external repositories follow their own branch and approval policy. Establish any permitted merge tools explicitly.
- Artifact history is kept by the plugin, but only the latest deployment is publicly hosted. Source downloads refer to that deployed revision.

Baseline references for technical scoping

Repository: github.com/zac-williamson/aztec_experiments. Source snapshot: master 458724c, inspected 11 October 2026. Read AGENTS.md, execution/README.md and TESTING.md before implementation.

- Merged feature evidence: docs/merges/2026-10-08/result.json and docs/merges/2026-10-08-plugins/result.json.
- V6 baseline and limits: package.json, installed @aztec-labs/constants source, execution/evidence/V06/acceptance.md and execution/evidence/V06.json.
- Existing service boundaries: prover/README.md, plugins/README.md, plugins/ESCROW_SPEC.md, docs/privacy.md and docs/recovery.md.
- Production qualification obligations: execution/requirements.md and the current execution graph. Historical evidence does not qualify newly changed source.